

```
[yogindar@yogindar-server1 test]$ ls
```

```
oldname.c oldname-test.c oldname.txt test-oldname.c
```

```
[yogindar@yogindar-server1 test]$ for file in `ls  
"oldname*"; do mv $file $(echo $file | sed -e "s/oldname/  
newname/g"); done
```

```
[yogindar@yogindar-server1 test]$ ls  
newname.c newname-test.c newname.txt test-newname.c
```

—Yogindar Das Yasodhar, yogindar@yahoo.com



Displaying a Perl script output in the Web browser

To display the script output, we need to install the following modules:

```
HTTP::Server::Simple::CGI
```

Use the following command to install them:

```
#sudo cpan -i HTTP::Server::Simple::CGI
```

Or, download the *tar* from *cpan* and then install it. The script is:

```
#!/usr/bin/perl  
{  
    package WebServer;  
  
    use HTTP::Server::Simple::CGI;  
    use base qw(HTTP::Server::Simple::CGI);  
  
    sub handle_request {  
        print "Hi this is Rupin\n";  
        $date = qx("date");  
        print "\nToday's date is - $date";  
        $ip = qx("ifconfig");  
        print "\n\n$ip";  
    }  
}  
  
# start the server on port 8080  
my $server = WebServer->new();  
$server->run();
```

Save the above script and run it. Now open the browser and type the following command:

```
http://localhost:8080
```

You will get the output in the browser.

—Rupin Puthukudi, rupinmp@gmail.com



How to check the number of protocol connections made in a server

Here is a simple tip to check the number of connections made by HTTP or MySQL.

We know port 80 is used by the HTTP server and port 3306 is used by the MySQL server, by default. So here is the command to check the number of connections:

```
# netstat -an | grep :80 | wc -l
```

The above command is for the HTTP server. Similarly, type 3306 instead of 80 to know the connection made for the MySQL server.

To know the IP addresses that have made requests to port 80 of your server, run the following command:

```
# netstat -an | grep :80
```

—Rajnish kumar, rajnishes91@gmail.com



Using *sudo* with a password in one line

We can avoid prompts for passwords by giving the *sudo* password as a standard input to the *sudo* command, as shown below:

```
$echo <password> | sudo -S <command>
```

For example:

```
[root@admin ~]$ echo 'mypassword' | sudo -S ./acc.out
```

Here, the echo 'mypassword' will send *mypassword* to the output, where | will send this as standard input to the *sudo* command run with the -S parameter.

-S is *-stdin*, which reads the password from the standard input.

—Pallavi Rawat, pallavifirst@rediffmail.com



Share Your Linux Recipes!

The joy of using Linux is in finding ways to get around problems—take them head on, defeat them! We invite you to share your tips and tricks with us for publication in *OSFY* so that they can reach a wider audience. Your tips could be related to administration, programming, troubleshooting or general tweaking. Submit them at www.opensourceforu.com. The sender of each published tip will get a T-shirt.